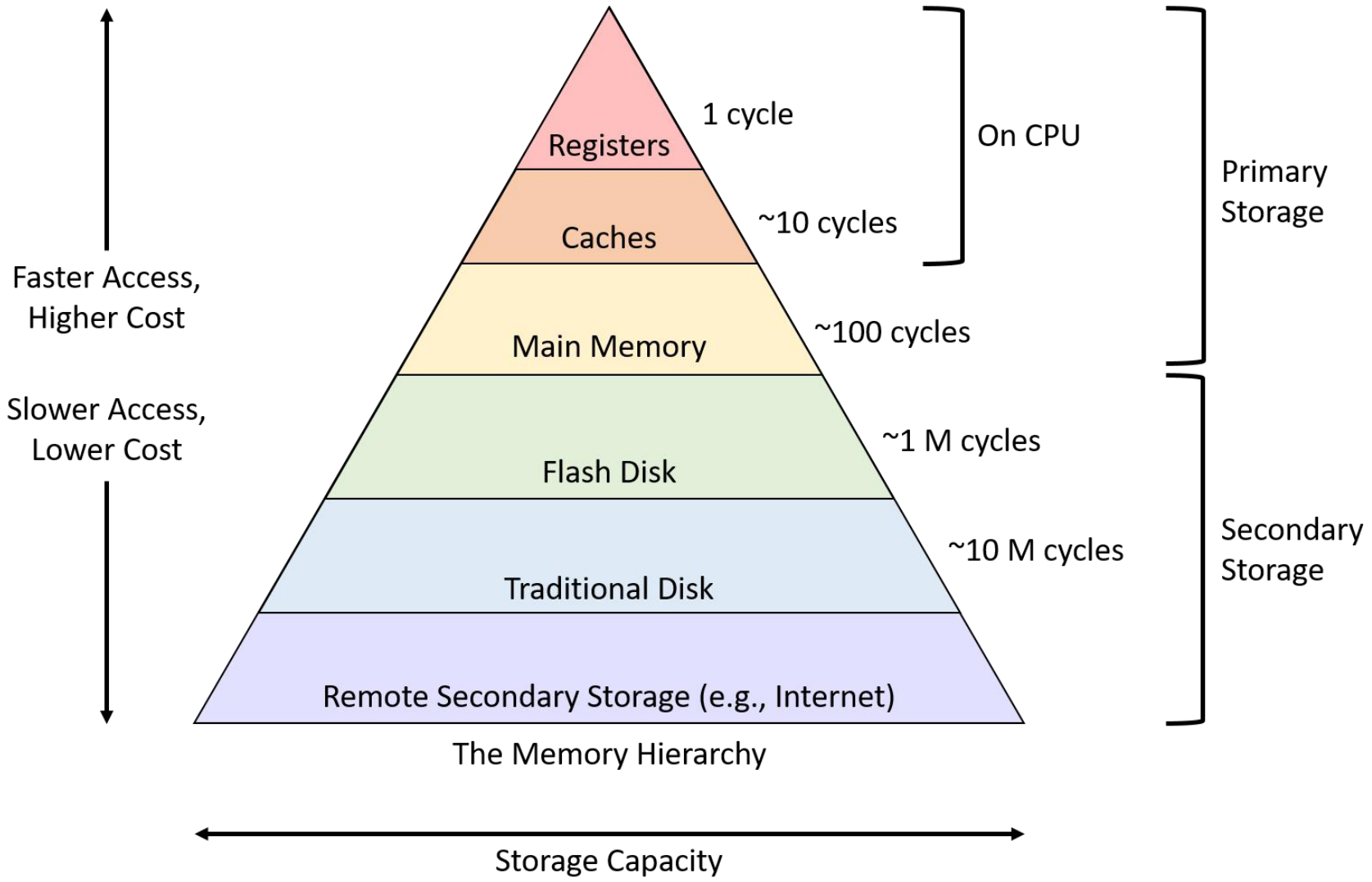
Memory: Cache, Disk and Virtual Memory

Outline

- Cache
- Disk
- Virtual Memory

Memory Hierarchy



Cache Basics

- The purpose of cache memory is to speed up accesses by storing some data closer to the CPU.
- Although cache is much smaller than main memory, its access time is a fraction of that of main memory.
- Unlike main memory, which is accessed by address, cache is typically accessed by content; hence, it is often called content addressable memory.

Cache memory and cache control unit

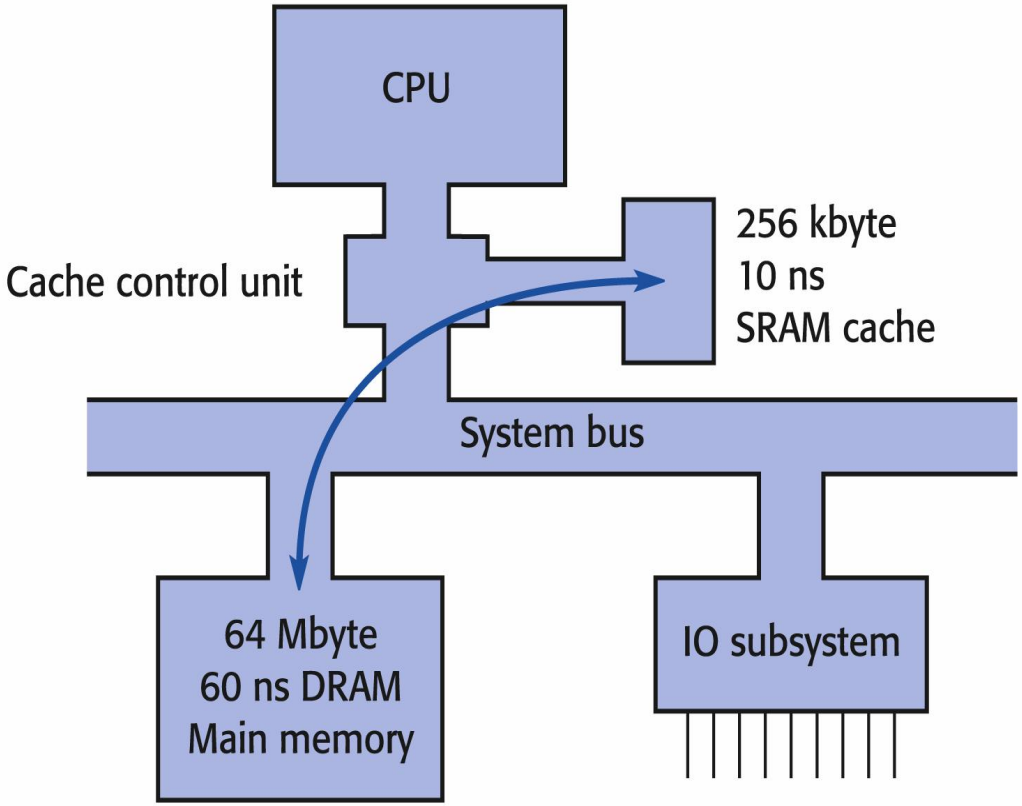
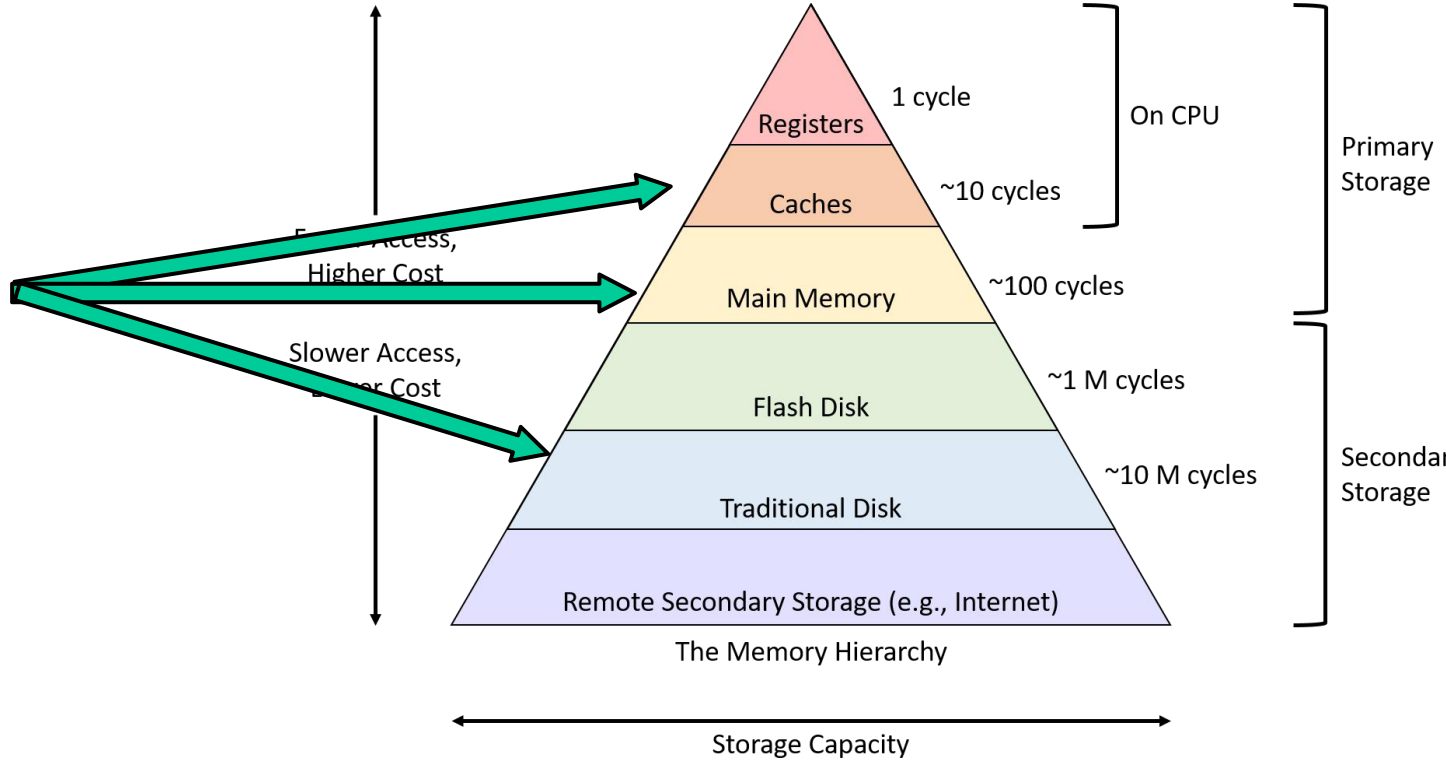


Fig. 12.9 Cache memory and controller unit.

© Pearson Education 2001

Cache memory and Main memory

```
int myarray[5];
myarray[0] = 1;
myarray[1] = 3;
myarray[2]= 5;
…
_asm {
…
}
```



Localisation of access

- The idea of cache memory exploits **Localisation of Memory Access** principle:
  - Computers tend to spend periods of time accessing the same locality of memory.
- Therefore:
  - A portion of code or data which require access needs to be loaded into the fastest memory nearest to CPU.
  - Other sections of the program and data can be held in readiness lower down the memory hierarchy.

Locality Example 1

Identify locality using the following program code:

```
int myarray[5]; // declaration of an array of integers
myarray[0] = 1; //
myarray[1] = 3; //
myarray[2] = 5; //
...
_asm {
lea ebx,myarray          //address of the array (its 0th element) is saved in ebx
mov ecx,5                // size of the array is saved in the counter
mov eax,0                // eax will be used to store the sum, initialise to 0
Loop1: add eax,[ebx]     //read an element of the array at the address stored in ebx
add ebx,4                //int occupies 4 bytes (32 bits),
                         //so to read next element increase an address in ebx by 4
loop Loop1               // the end of the loop
...
}
```

Why localisation of access works?

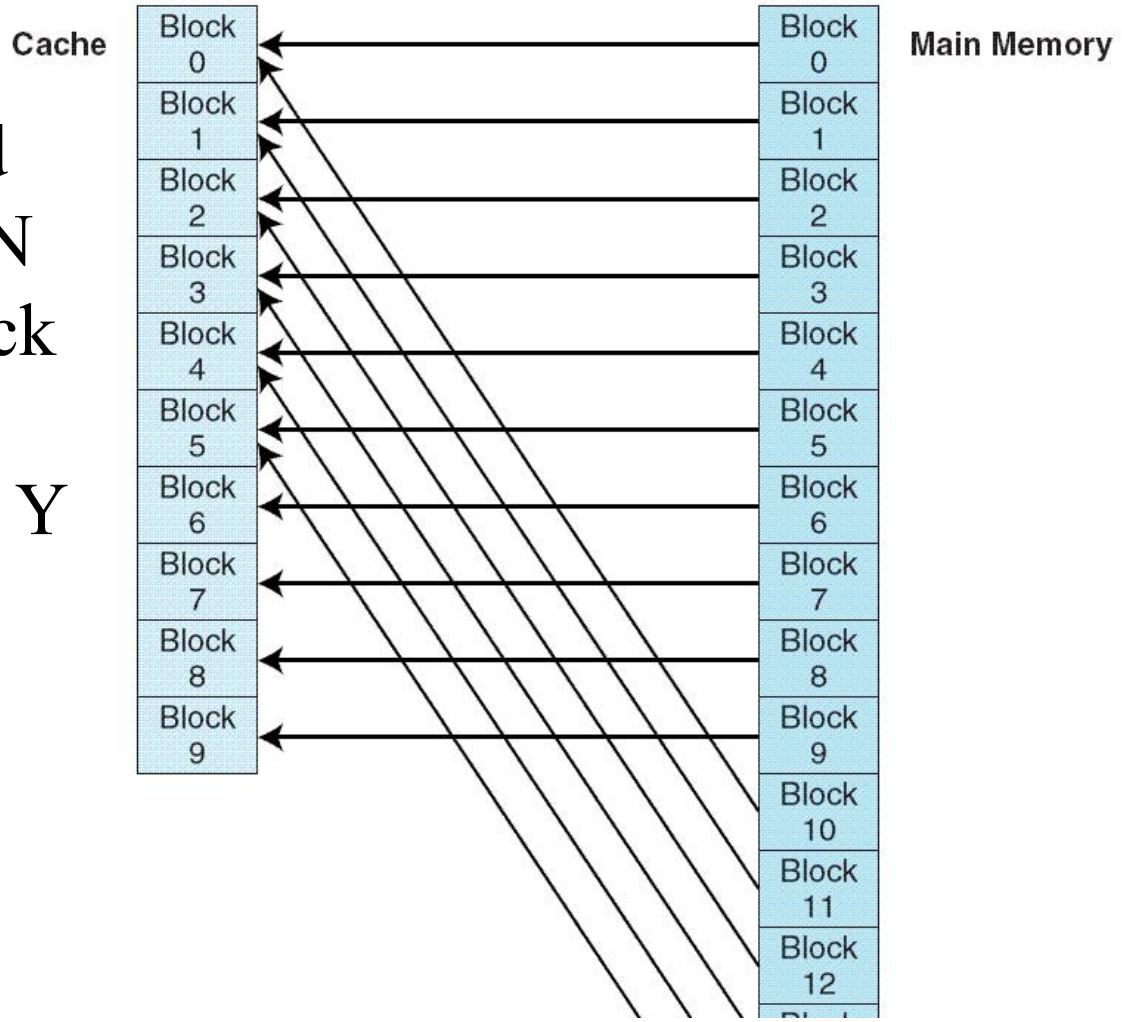
- Partly due to the programmer clustering related data items together in arrays or records.
- Partially due to the repeating patterns in a program (i.e. loops)
- Partially due to the compiler attempting to organise the code in an efficient manner.

Cache Field

- The “content” that is addressed in cache is a subset of the bits of a main memory address called a field.
  - Many blocks of main memory map to a single block of cache.
  - A tag field in the cache block distinguishes one cached memory block from another.
  - A valid bit indicates whether the cache block is being used.
  - An offset field points to the desired data in the block.

Cache Field Example

•With direct mapped cache consisting of N blocks of cache, block X of main memory maps to cache block Y = X mod N.



Cache Field Example

- A simple cache mapping scheme is direct mapped cache.
  - In a direct mapped cache consisting of N blocks of cache, block X of main memory maps to cache block Y = X mod N.
  - Thus, if we have 10 blocks of cache, block 7 of cache may hold blocks 7, 17, 27, 37, . . . of main memory.

Cache Update

- A replacement policy is invoked when it is necessary to evict a block from cache.
- An optimal replacement policy would be able to look into the future to see which blocks won't be needed for the longest period of time.
- A benchmark is needed for assessing the efficiency of replacement policy .

Cache Update

- The replacement policy that we choose depends upon the locality that we are trying to optimize.
- A least recently used (LRU) algorithm keeps track of the last time that a block was assessed and evicts the block that has been unused for the longest period of time.
  - The disadvantage of this approach is its complexity: LRU has to maintain an access history for each block, which slows down the cache.

Cache Update

- First-in, first-out (FIFO) is a popular cache replacement policy.
  - The block that has been in the cache the longest, regardless of when it was last used.
- A random replacement policy does what its name implies: It picks a block at random and replaces it with a new block.
  - Random replacement can certainly evict a block that will be needed often or needed soon, but it never thrashes.

Performance of Caching

- Performance of Caching depends upon programs' locality.
  - Some programs may have poor locality owing to their complex, dynamic structures.
- With poor locality, caching can actually cause performance degradation rather than performance improvement.

Performance of Caching (cont.)

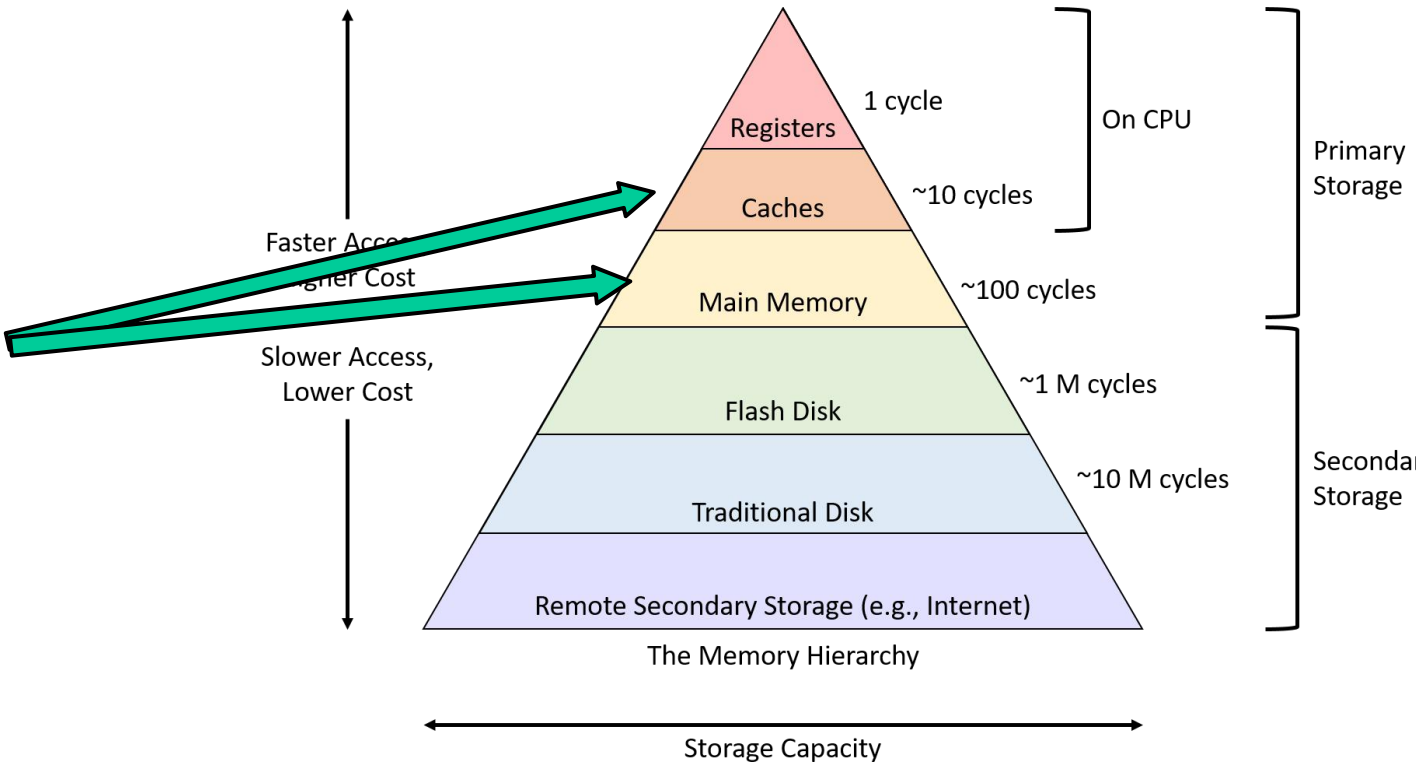
- Which one may have a poor locality

```
int myarray[1000000];
myarray[0] = 1;
myarray[1] = 3;
myarray[2]= 5;
...
_asm {
...
}
```

```
int myarray[1000000];
myarray[0] = 1;
myarray[100000] = 3;
myarray[200000]= 5;
...
_asm {
...
}
```

Dirty Blocks

```
int myarray[5];
myarray[0] = 1;
myarray[1] = 3;
myarray[2]= 5;
...
_asm {
...
}
```



Dirty Blocks

- Blocks that have been updated in the cache, but not in the main memory
  - Blocks must be written back to memory. A write policy determines how this will be done.
  - There are two types of write policies, write through and write back.
  - Write through updates cache and main memory simultaneously on every write.
  - Write back delays updates until the dirty block is pushed out or replaced to save time

Discussion

- The pros and cons of write through and write back

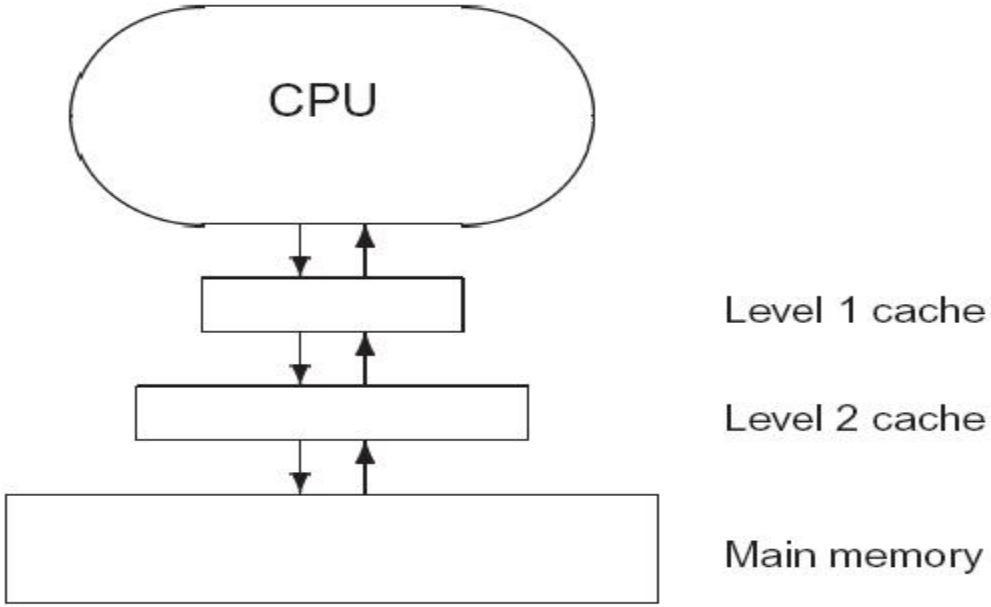
Separate Caches

- The cache we have been discussing is called a unified or integrated cache.
  - both instructions and data are cached.
- Many modern systems employ separate caches for data and instructions.
  - This is called a Harvard cache.
- The separation of data from instructions provides better locality, at the cost of greater complexity.

Levels of cache

- Most of today's small systems employ multilevel cache hierarchies.
  - Cache form their own small memory hierarchy.
  - Level1 cache (8KB to 64KB) is situated on the processor itself.
  - Access time is typically about 4ns.
  - Level 2 cache (64KB to 2MB) may be on the motherboard, or on the processor itself.
  - Access time is usually around 15 - 20ns.

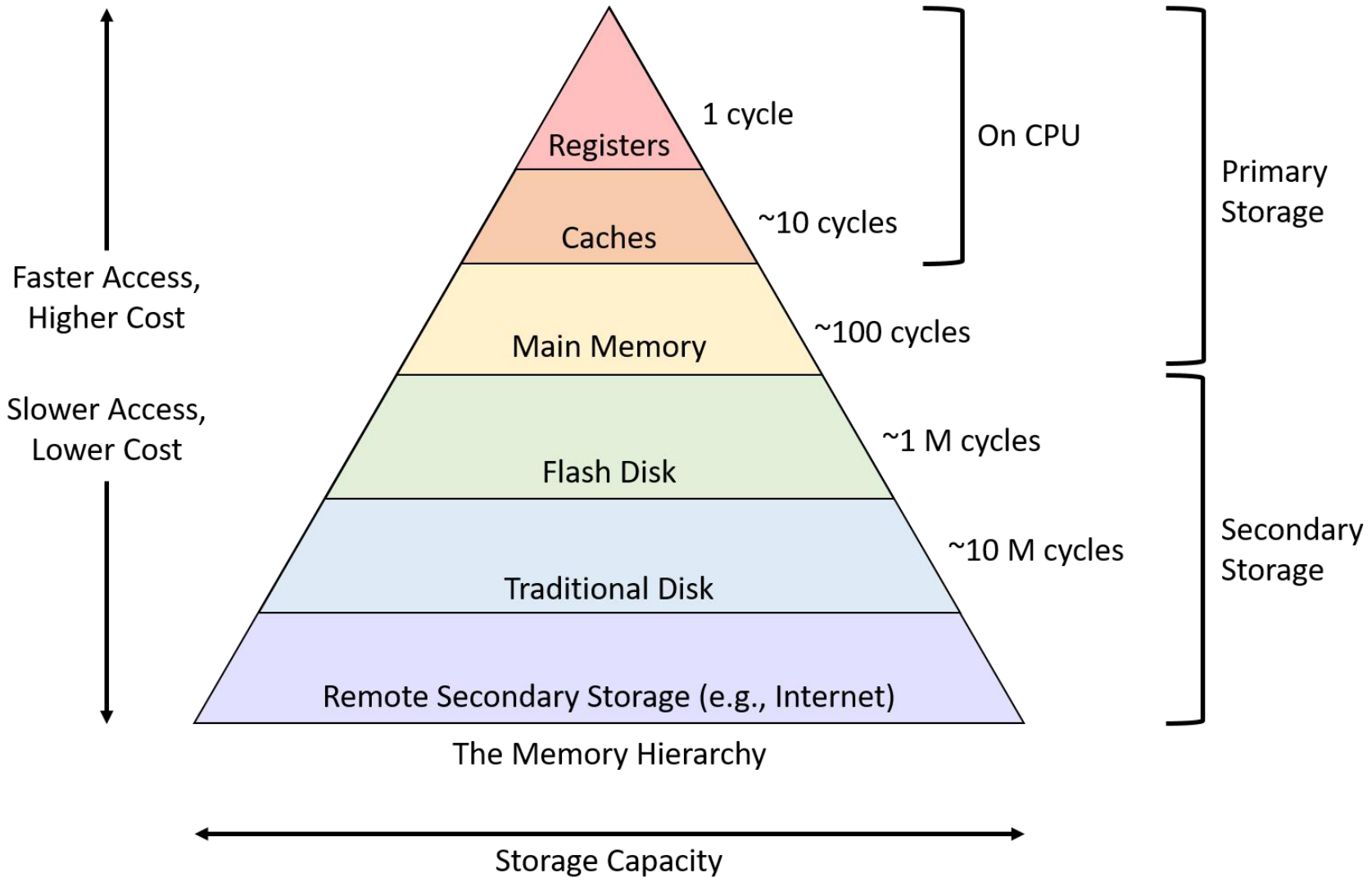
Levels of cache



Inclusive vs Exclusive Cache

- In inclusive cache, the same data may be present at multiple levels of cache.
  - Strictly inclusive caches guarantee that all data in a smaller cache also exists at the next level.
- Exclusive caches permit only one copy of the data.
- The tradeoffs in choosing one over the other involve weighing the variables of access time, memory size, and circuit complexity.

Memory Hierarchy



Mass storage

- It refers to various techniques and devices for storing a large amount of data.
- Unlike main memory, mass storage devices retain data even when the computer is turned off.

Types of mass storage

- Hard disks drives (HDD).
- Solid State Drives (SSD).
- Optical disks: CD-ROM, DVD.
- USB disks.
- Etc.

HDD Basics

- Hard disk drives are the most important types of permanent storage.

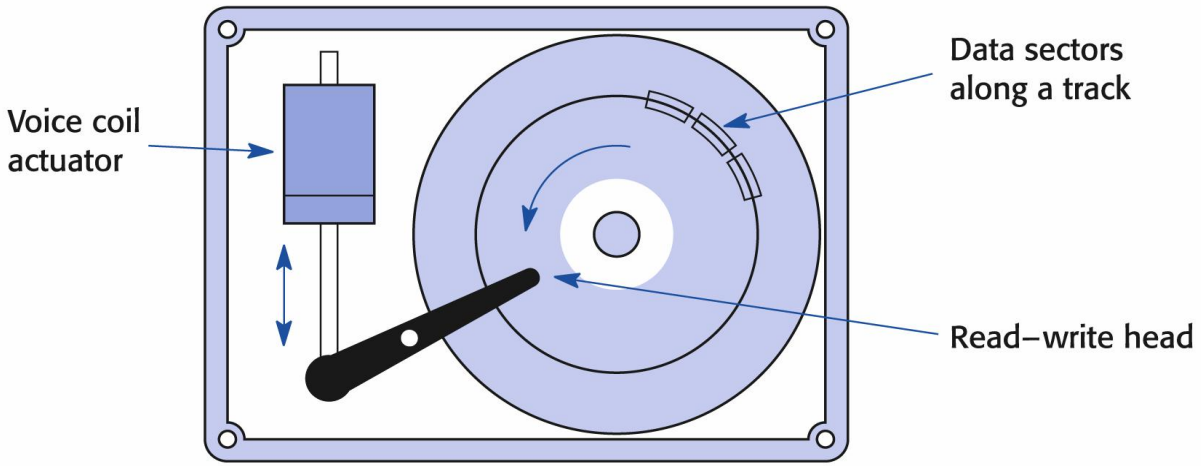
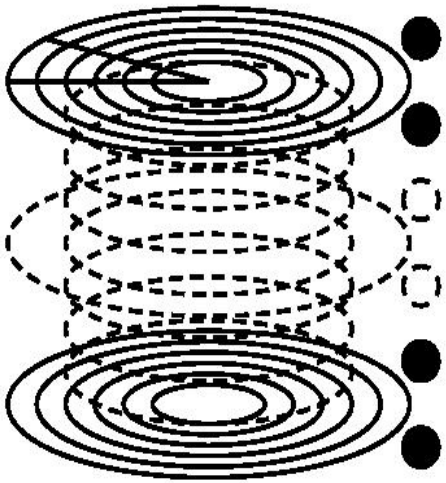


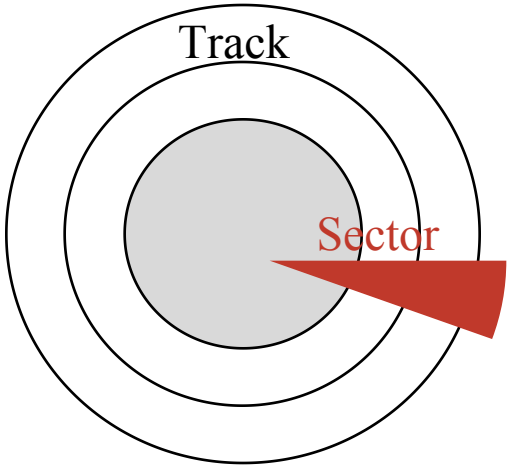
Fig. 12.18 Schematic diagram of hard disk unit.

© Pearson Education 2001

HDD Basics

- Hard disk drives are the most important types of permanent storage.
  - sevelral paltters, with information recorded on both surfaces
  - 500 to 2,000 tracks per surface
  - 32 to 128 sectors per track
  - A sector is the smallest unit that can be read or written



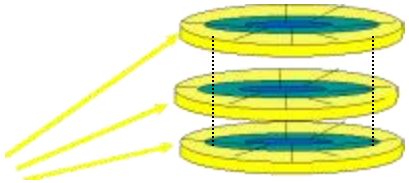


Platters *

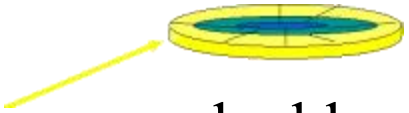
Disk cylinders, tracks, sectors



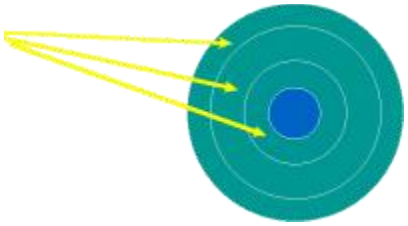
hard disk



cylinders



double sided floppy



tracks



sectors

Addressing in HDD

- **CHS**.
  - **C**ylinder, **H**ead, **S**ector system.
  - Telling the disk controller which cylinder, head and sector to access.
  - Can be mapped onto LBA.
- **LBA**.
  - **L**arge **B**lock **A**ddressing, by absolute number of a sector.

Addressing in HDD (cont.)

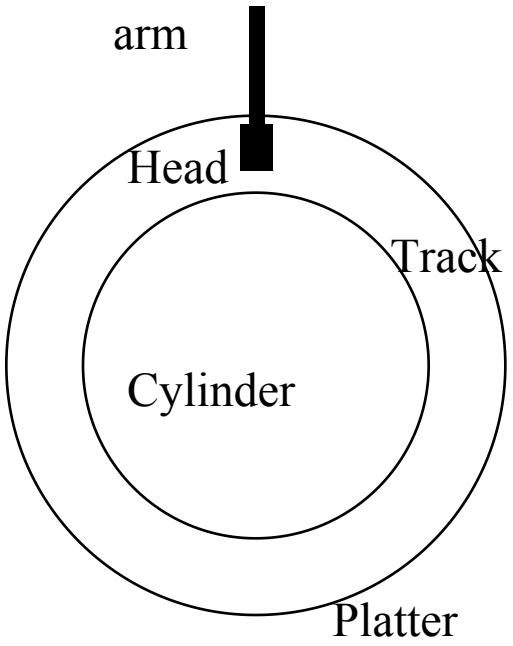
- **Example (simplified)**:

A piece of information needs to be read.

1. The first step is to figure out where on the disk to look for the needed information.
2. The location on the disk → address expressed either:
   - In terms of the **cylinder, head and sector** (CHS).
   - Or, in terms of the **absolute sector number** (LBA).
3. A request is sent to the drive over the **disk drive interface** giving it this address and asking for the sector to be read.

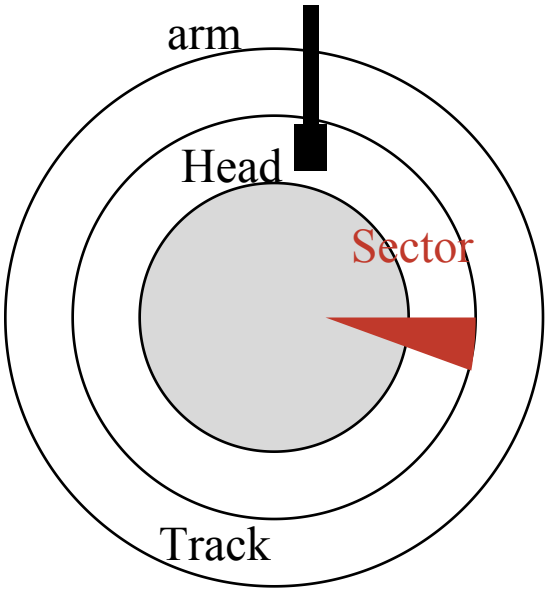
HDD Read/Write

- Read/write data is a process:
  - figure out where on the disk to look for the data. Address expressed either:
    - head and sector (CHS).
    - absolute sector number (LBA).
  - position the arm over the proper track, known as seeking
  - wait for the desired sector to rotate under the read/write head
  - transfer a block of bits (sector) under the read-write head



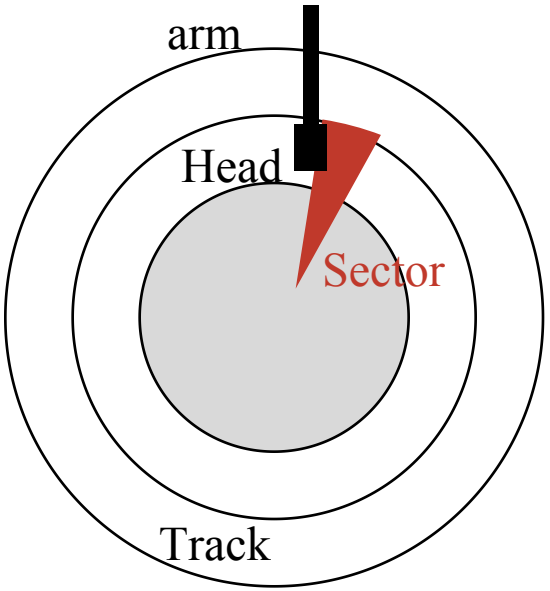
HDD Read/Write

- Read/write data is a process:
  - figure out where on the disk to look for the data. Address expressed either:
    - head and sector (CHS).
    - absolute sector number (LBA).
  - **position the arm over the proper track, known as seeking**
  - wait for the desired sector to rotate under the read/write head
  - transfer a block of bits (sector) under the read-write head



HDD Read/Write

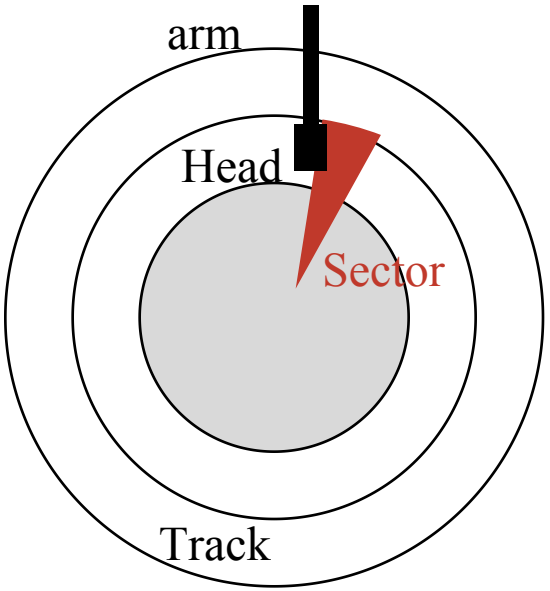
- Read/write data is a process:
  - figure out where on the disk to look for the data. Address expressed either:
    - head and sector (CHS).
    - absolute sector number (LBA).
  - position the arm over the proper track, known as seeking
  - **wait for the desired sector to rotate under the read/write head**
  - transfer a block of bits (sector) under the read-write head



*

HDD Read/Write

- Read/write data is a process:
  - figure out where on the disk to look for the data. Address expressed either:
    - head and sector (CHS).
    - absolute sector number (LBA).
  - position the arm over the proper track, known as seeking
  - wait for the desired sector to rotate under the read/write head
  - **transfer a block of bits (sector) under the read-write head**



Read/Write Time

- Access time = queue + seek + rotational + transfer + overhead
- Queue: disk might be busy
- Seek time: move arm over track
- Rotational latency
  - wait for sector to rotate under head
  - average = 0.5/(3600 RPM) = 8.3ms
- Transfer Time
  - f(size, BW bytes/sec)

Read/Write Time Example

- Assume the a disk's overhead in figuring out address is 2 ms. The disk is idle so no queuing delay. The disk parameters are below. What is Average Disk Access Time for a Sector?
  - Transfer size is 8K bytes
  - Average seek time is 12 ms
  - Disk spins at 7200 RPM
  - Transfer rate is 4 MB/sec

Ans: Ave seek + ave rot delay + transfer time + overhead

Progress for the last 25 years

- First PC hard disks: capacity 10 MB and cost about 100 pounds (RMB$1100) per 1 MB.
- Now: capacity > 300 GB and cost < half penny for 1 MB.

*

Flash Basics

- Built from NAND flash: electrically erasable, non-volatile memory cells
- No moving parts — unlike a spinning hard disk, there's no platter or read/write arm
- Data is stored as trapped electric charge on a floating-gate transistor
- Retains data with power off - same durability guarantee as a hard disk

*

Flash vs HDD

- Random read latency: 0.1 ms (~5–10 ms for HDD)
- I/O operations: 500MB/s on a modern SSD, and 80~100MB/s for HDD
- Costs roughly 4~6× more per GB than a traditional hard disk today

Disk cache

- A portion of main memory used as a **buffer** to temporarily hold data for the disk.
- Disks write operations are ***clustered***. (Why?)
- Some data written out may be needed again.
  - The data are retrieved rapidly from the disk cache instead of slowly from disk.
- Java "`flush()`" in `java.io.PrintWriter`.

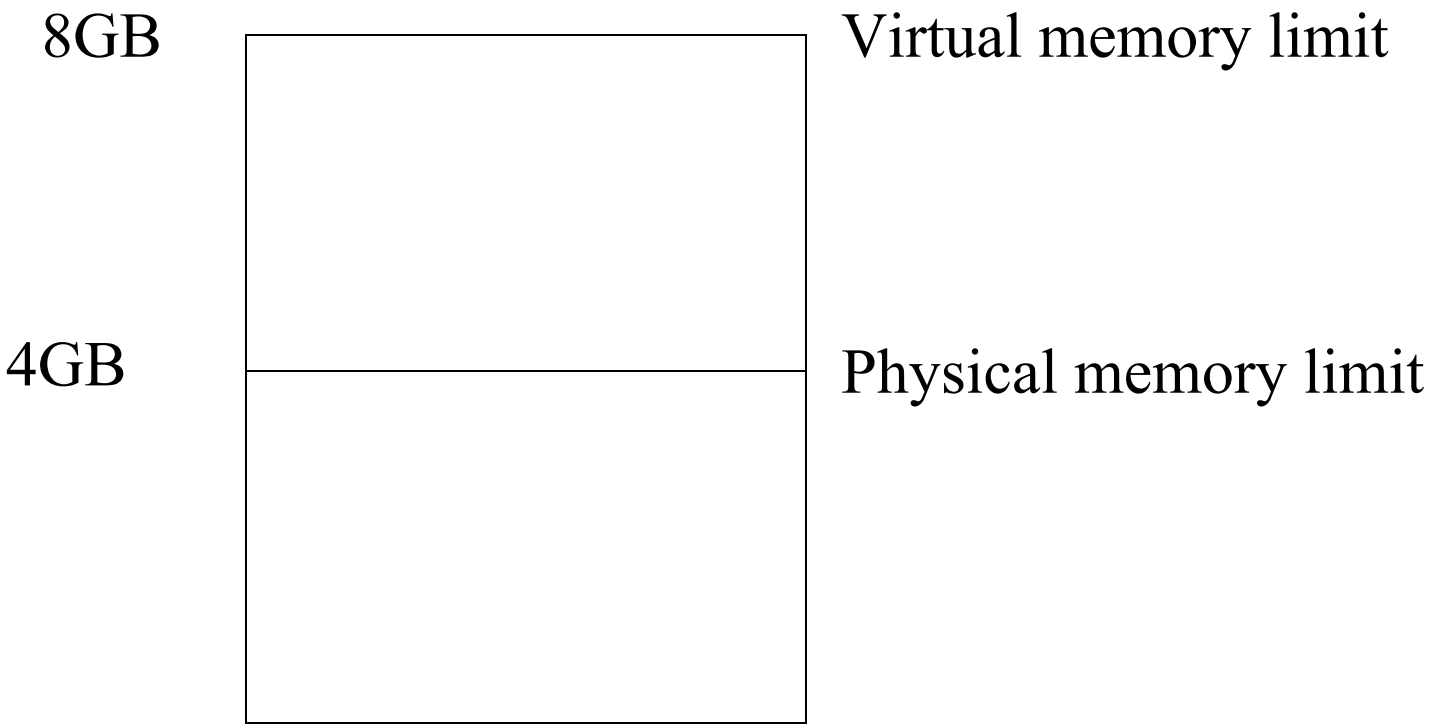
Read and Write

- Retrieving files into RAM is called reading
  - loading an application
  - opening a file
- Copying data from RAM onto a secondary storage device is called writing

Virtual memory

- Virtual memory is a technique, in a sense, opposite to caching:
  - It is the use of low-level memory (i.e. hard disk) to ‘expand’ high-level (main) memory.
  - It provides a convenient expansion of main memory by ‘overflowing’ data and program code onto magnetic disk.
- The area **on disk** reserved for this purpose is known as the **swap area**.

Virtual Memory



*

Memory Management



- Virtual memory
  - hard disk space
  - when processor needs more RAM space, swaps unused data onto designated hard disk space
  - improves flexibility but is slower than RAM to which the processor has direct access

Remote Storage

- Data lives on disks in a data center you don't own, accessed over a network
  - Examples: Microsoft OneDrive, Amazon S3, Google Drive, Dropbox, Baidu NetDisk,
- Backed by massive arrays of hard disks across many machines
  - Data is replicated across multiple disks, racks, and often geographic regions

Remote Storage Features

- Cheapest tier per GB - often a fraction of a cent per GB per month for cold storage
- High and variable latency in memory hierarchy
  - Improve latency with caching and CDNs by keeping a copy of “hot” data close to user
- Best suited for
  - backups, archives, big data, collaborative files, static web content

Q&A

- Q. Given the following components, form a reasonable memory hierarchy to ensure good cost-speed tradeoff.
- Cache, disk, RAM

Q&A

- Q. Locality can be temporal, which means data that are recently used tend to be accessed more likely than the others. (T or F)

Q&A

- Q: When you save to this, your data will remain intact even when the computer is turned off.
- a. RAM
- b. motherboard
- c. secondary storage device
- d. primary storage device

Q&A

- Q: The closed, concentric rings on a diskette are referred to as
- a. grooves.
- b. tracks.
- c. sectors.
- d. circles.

*